

What is Memoization? in JavaScript

What is Memoization?

- Memoization is an optimization technique used to speed up expensive function calls by caching their results.
- Every time a function runs with the same input, it returns the stored result instead of recalculating it from scratch.

Why is Memoization Important?

- Speeds up performance for recursive or heavy computations.
- Great for interview questions like:
 - Fibonacci
 - Factorial
 - Dynamic programming
- Used in real-world apps: Handling React re-renders, API calls caching, etc.

Without Memoization



```
function fib(n) {  
  if (n <= 1) return n;  
  return fib(n - 1) + fib(n - 2); // 🐌 Slow  
}  
console.log(fib(30)); // ⌚ Takes a lot of time
```

Problem: It makes unnecessary repeated calls by re-computing the same inputs over and over.

With Memoization



```
function memoizedFib() {  
  const cache = {};  
  return function fib(n) {  
    if (n in cache) return cache[n]; // Returns cached result  
    if (n <= 1) return n;  
    cache[n] = fib(n - 1) + fib(n - 2);  
    return cache[n];  
  };  
}  
  
const fib = memoizedFib();  
console.log(fib(30)); // 🚀 Much faster
```

- ✓ Caches results.
- ✓ Avoids duplicate calculations.
- ✓ Reduces time complexity significantly.

Memoization Template



```
function memoize(fn) {  
  const cache = {} ;  
  return function (...args) {  
    const key = JSON.stringify(args) ;  
    if (key in cache) return cache[key] ;  
    const result = fn(...args) ;  
    cache [key] = result ;  
    return result ;  
  };  
}
```

Example Use Case – Factorial



```
const factorial = memoize (function (n) {  
  if (n === 0) return 1;  
  return n * factorial (n - 1);  
});
```

```
console.log(factorial(5)) ; //  Result is Cached
```

When to Use Memoization?

- ✅ Heavy recursive functions: Where the same sub-problems are solved multiple times.
- ✅ Expensive calculations: Functions that take a lot of processing time.
- ✅ Pure Functions: Functions that always give the same output for the same input.
- ⚠️ Avoid if: The function has side effects or involves randomness.